Explore Functional Programming in Go

มองโค้ดให้เห็น 3 อย่าง: Actions · Calculations · Data

ทักษะที่ใช้กับโค้ด Go ที่ทำงานจริงได้ทันที — ไม่ต้องมีพื้น FP มาก่อน

จบ 2 ชั่วโมงนี้ คุณจะได้

เรียงจาก 'มองเป็น' → 'เขียนเป็น' → 'ใช้เป็น'

- ทักษะมองโค้ด — เห็นว่าบรรทัดไหนเป็น Action / Calculation / Data แล้วรู้ว่ามันแปลว่าอะไร
- วิธีดึง calculation ออกจาก action ให้โค้ดทดสอบง่ายขึ้น — ใช้กับ legacy Go ได้ทันที
- เครื่องมือของ Go สำหรับเขียนแบบ functional: closures, generics, iter.Seq
- สายตาใหม่กับ pattern ที่คุณใช้อยู่แล้ว (Functional Options, middleware) — ที่จริงมันคือ FP
- **และมีการบ้าน 1 ชิ้น ให้ได้ลอง refactor ด้วยมือตัวเองต่อที่บ้าน**

แกนคิดของวันนี้: ทุกบรรทัดโค้ดคือ 1 ใน 3 อย่างนี้

	คืออะไร	ตัวอย่าง	ชื่อ FP อื่นเรียก
Data	ข้อเท็จจริงของเหตุการณ์	email ที่ user กรอก, ยอดเงินจาก API	—
Calculation	input เดิม → output เดิม เสมอ	หาส่วนลด, validate email	pure function
Action	ผลขึ้นกับ 'รันกี่ครั้ง / เมื่อไหร่'	ส่ง email, เขียน DB, อ่านเวลา	side effect

- **คำถามเดียวที่ต้องตอบเก่ง: "บรรทัดนี้พึ่งโลกภายนอกไหม?"**
- อ่านนาฬิกา, พิมพ์ log, เขียนไฟล์ → พึ่ง (ผลเปลี่ยนตามเวลา/จำนวนครั้ง)
- บวกเลข, ตรวจ format, หาคูปองที่ใช้ได้ → ไม่พึ่ง (input ชุดเดิม = คำตอบชุดเดิม)

ลองอ่านโค้ดนี้

บรรทัดไหนคือ Data · Calculation · Action? — บรรทัดเดียวซ้อนหลายอย่างได้

```
func ProcessOrder(order *Order) {
    order.ProcessedAt = time.Now()
    fmt.Printf("processing %s ...", order.Email)
    order.Total = 0
    for _, it := range order.Items {
        order.Total += it.Price * it.Quantity
    }
    if order.Total >= 1000 {
        order.Total = order.Total * 90 / 100
    }
    orders[order.Email] = *order
    fmt.Printf("saved: total=%d", order.Total)
}
```

ชวนคิด

- ทำไมฟังก์ชันนี้รัน 2 ครั้ง ผลอาจไม่เหมือนกัน?
- ถ้ากติกาส่วนลดเปลี่ยน ต้องรันระบบทั้งระบบเพื่อทดสอบไหม?
- ข้อมูล (Data) ที่ไหลผ่านฟังก์ชันนี้มีอะไรบ้าง?

โค้ดที่ปนกันทำให้ชีวิตยาก

ตัวอย่างเดิม — มองผ่านกรอบ ACD แล้วปัญหาชัดขึ้นเอง

- **Action แฝงใน calculation → ทดสอบยาก**
  - อยากทดสอบแค่กติกาส่วนลด ต้องรัน ProcessOrder ทั้งฟังก์ชัน — จับ stdout, ล้าง map, ปลอมเวลา
- **Mutate input → ผู้เรียกไม่รู้ว่า order ของตัวเองถูกแก้ไปแล้ว**
  - bug ยอดฮิต: ผ่าน order เข้าไป 2 ครั้ง ยอดหายเพราะถูกลดสองรอบ
- **กติกาธุรกิจ (ลด 10% เมื่อยอดถึง 1,000) ฝังอยู่ใน action → มองไม่เห็นเป็น 'ความรู้'**
  - กติกาพวกนี้คือ calculation ที่ควรรวมอยู่ด้วยกัน เทสต์ได้เอง และใช้ซ้ำได้

จำไว้: โค้ดที่ปนกันยังทำงานได้ — แต่คุณจ่ายค่ามันตอนเทสต์ ตอนแก้ และตอนเจอ bug

จาก Action สู่ Calculation

หัวใจของวันนี้ — แยก 'การคำนวณ' ออกจาก 'การทำลงโลกภายนอก'

ปนกัน

```
// ฟังก์ชันนี้ "คำนวณ" หรือ "ทำอะไรลงโลกภายนอก"?
func ApplyDiscount(cart Cart) Cart {
    fmt.Println("applying discount")
    cart.Total = cart.Total * 90 / 100
    return cart
}
```

แยกแล้ว

```
// คำนวณอย่างเดียว — input เดิม ได้ผลเดิมเสมอ
func Discounted(total int) int {
    return total * 90 / 100
}

// การ "ทำลงโลกภายนอก" แยกเป็นอีกฟังก์ชัน
func LogDiscount(total int) {
    fmt.Println("discount", Discounted(total))
}
```

- Discounted เป็น calculation แท้: ไม่พิมพ์ ไม่แก้ input ไม่ถามเวลา
- action ไม่หายไปไหน — ถูก 'ย้ายไปไว้ที่ขอบ' ของระบบ ตรงที่ต้องเกิดจริงเท่านั้น

จำไว้: แยกแล้วง่ายขึ้น: calculation เทสต์เองได้ · action เหลือบาง ๆ จุดเดียว

ผลลัพธ์: ทดสอบได้โดยไม่ต้อง mock อะไรเลย

```
func TestDiscounted(t *testing.T) {
    tests := []struct{ in, want int }{
        {1000, 900}, {999, 999}, {0, 0},
    }
    for _, tt := range tests {
        if got := Discounted(tt.in); got != tt.want {
            t.Errorf("Discounted(%d) = %d; want %d",
                tt.in, got, tt.want)
        }
    }
}
```

- เทียบกับตอนก่อน: เทสต์ ApplyDiscount ต้องจับ stdout + เช็คว่า input โดนแก้ — เทสต์กลายเป็นงานโบราณคดี
- calculation ที่ pure = เทสต์เป็นตาราง input → output ได้เลย — กติกาธุรกิจทั้งหมดอยู่ตรงนี้

จำไว้: อยากรู้ว่าโค้ดส่วนไหน 'ควรแยก' — ดูว่าการเทสต์มันต้อง mock อะไร ยิ่งเยอะ ยิ่งแปลว่า action ปนอยู่

เครื่องมือ 1: ฟังก์ชันเป็นค่า (Higher-order Functions + Closures)

Go ส่งฟังก์ชันเป็น argument / คืนฟังก์ชันได้เต็มรูปแบบ — std lib ใช้เยอะแล้ว: sort.Slice, http.HandleFunc

```
// ฟังก์ชันคืนฟังก์ชันได้ — "เก็บครึ่งการคำนวณ" ไว้ก่อน
func Adder(a int) func(int) int {
    return func(b int) int { return a + b }
}

add10 := Adder(10)
add10(5) // == 15

// generic: curry ฟังก์ชัน 2 พารามิเตอร์
func Curry2[A, B, R any](f func(A, B) R) func(A) func(B) R {
    return func(a A) func(B) R {
        return func(b B) R { return f(a, b) }
    }
}
```

จำไว้: closure จำบริบทตอนถูกสร้าง นี่คือวิธี Go ทำ partial application (verbose กว่าภาษา ML ชัดเจน — และนี่คือเหตุผลที่ Go ไม่มี pipe operator)

กับดักของ closure: loop variable

closure จับ 'ตัวแปร' ไม่ใช่ 'ค่า' — เว้นแต่ Go 1.22 เป็นต้นไป

```
for i := 0; i < 3; i++ {
    go func() { fmt.Println(i) }()
}

// Go 1.22+: พิมพ์ 0..2 ไม่ซ้ำ
//          (loop var แยกของตัวเองทุกรอบ)
// Go 1.21 ลงไป: ส่วนใหญ่พิมพ์ 3 3 3
//          (ทุก closure จับ "ตัวแปรตัวเดียวกัน")

// โค้ดเก่าจึงมีรอยแก้แบบนี้ทิ้งไว้:
for i := 0; i < 3; i++ {
    i := i // ทำสำเนาต่อรอบ
    go func() { fmt.Println(i) }()
}
```

จำไว้: โค้ดเก่าก่อน 1.22 ยังมีรอยแก้ i := i ทิ้งไว้ — อ่านเจอแล้วอย่าลบทิ้งโดยไม่รู้ว่ามันกันอะไร

เครื่องมือ 2: Generics — เขียน Map / Filter / Reduce เองได้

Go 1.18+ · โค้ดเต็ม + เทสต์แบบ TDD ใน go-fp/03-fp-toolkit

```
func Map[T, R any](in []T, f func(T) R) []R {
    out := make([]R, 0, len(in))
    for _, v := range in { out = append(out, f(v)) }
    return out
}

func Filter[T any](in []T, keep func(T) bool) []T { ... }

// accumulator ต่าง type กับสมาชิกได้
func Reduce[T, R any](in []T, init R,
    combine func(R, T) R) R { ... }

total := Reduce(items, 0, func(acc int, it Item) int {
    return acc + it.Price*it.Quantity
})
```

- Go ไม่มี built-in map/filter — และ Go community ก็นิยม for loop ตรง ๆ
- โค้ดชุดนี้คือเครื่องมือ 'เรียนรู้ concept' — ในงานจริงใช้เมื่อมันทำให้เจตนาของโค้ดอ่านออกง่ายขึ้น

จำไว้: Reduce ที่ accumulator ต่าง type กับสมาชิก (Reduce[Item, int]) คือจุดที่ generics ช่วยจริง

Functional Options — pattern ที่คุณใช้อยู่แล้ว

closure + higher-order function ในโค้ด production จริง: grpc-go, zap, aws-sdk-go-v2

```
type Option func(*Server)

func WithPort(p int) Option {
    return func(s *Server) { s.port = p }
}

func New(host string, opts ...Option) *Server {
    s := &Server{host: host, port: 8080} // default ก่อน
    for _, o := range opts { o(s) }      // แล้ว override
    return s
}

srv := New("localhost", WithPort(9090))
```

- WithPort(9090) คือ 'ฟังก์ชันที่คืน action เล็ก ๆ' เอาไว้ยิงใส่ constructor ตอนหลัง
- ทำไมไม่ใช้ struct config? — options ขยายได้โดยไม่ทำ breaking change และ default ชัดเจน

จำไว้: คุณเขียน FP อยู่แล้วโดยไม่รู้ตัว — งานวันนี้แค่ตั้งชื่อให้สิ่งที่ทำอยู่

Middleware = Function Composition

http.Handler เป็น function เดียว → wrapper ครอบซ้อนกันคือ composition แบบ Go

```
type Middleware func(http.Handler) http.Handler

func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s (%s)", r.Method, r.URL, time.Since(start))
    })
}

// เอา middleware มาครอบซ้อนกัน = composition
func Chain(mw ...Middleware) Middleware {
    return func(final http.Handler) http.Handler {
        for i := len(mw) - 1; i >= 0; i-- {
            final = mw[i](final)
        }
        return final
    }
}
```

จำไว้: ลำดับของ Chain(Logger, Recover) มีความหมาย — composition คือ 'ลำดับ = ความหมาย'

ของหวาน: Lazy Sequence ด้วย iter.Seq (Go 1.23+)

อนุกรมไม่รู้จบที่คำนวณเท่าที่ถูกขอ — range-over-func ในภาษา Go เอง

```
// อนุกรมไม่รู้จบ — ไม่มีเงื่อนไขหยุดในตัว
func Fibonacci() iter.Seq[int] {
    return func(yield func(int) bool) {
        a, b := 0, 1
        for {
            if !yield(a) { return } // ผู้อ่านเป็นคนสั่งหยุด
            a, b = b, a+b
        }
    }
}

isEven := func(n int) bool { return n%2 == 0 }
even := Filter(Fibonacci(), isEven)
total := 0
for n := range Take(10, even) { // คำนวณเท่าที่ขอเท่านั้น
    total += n
} // sum แรก 10 ตัวเลขคู่ = 257114
```

จำไว้: Fibonacci ไม่มีจุดจบ แต่ Take(10) คำนวณแค่ 10 ตัว — lazy คือคำนวณ 'เท่าที่จำเป็น' ไม่ใช่ 'ทั้งหมดก่อนแล้วค่อยตัด'

สรุป: 3 คำถามกับโค้ดทุกบรรทัด

ถาม	ถ้าคำตอบคือ...	แล้วทำอะไร
"ผลขึ้นกับเวลา/จำนวนครั้งไหม?"	ใช่ → Action	ย้ายไปไว้ขอบระบบ ทำให้บางที่สุด
"input เดิม ให้ผลเดิมเสมอไหม?"	ใช่ → Calculation	ทำให้ pure + เขียน table-driven test
"เป็นแค่ข้อเท็จจริงที่ส่งต่อกันไหม?"	ใช่ → Data	ใช้ immutable เท่าทำได้ (value semantics)

- Go ทำได้ดี: HOF · closures · generics (1.18+) · iter.Seq (1.23+)
- Go ไม่มี: built-in map/filter · immutability guarantee · tail-call optimization — รู้ขีดจำกัด แล้วออกแบบรอบมัน

จำไว้: ไม่ต้องเขียน FP ทั้งวัน — แค่หมั่นย้าย calculation ออกจาก action ทีละฟังก์ชัน

การบ้าน: แยกคูปองออกจากการส่งอีเมล

โจทย์จาก Grokking Simplicity บท 3–5 พอร์ตเป็น Go — repo: go-fp/07-coupon-email-exercise (มี EXERCISE.md ประกอบ)

starter/coupons.go — จุดตั้งต้น

```
// ทุกอย่างอยู่ในฟังก์ชันเดียว — มองแล้วเจออะไรบ้าง?
func EmailCoupons(cart Cart, coupons []Coupon,
    out io.Writer) error {
    fmt.Printf("finding coupons for %s\n", cart.Email)
    var codes []string
    for _, c := range coupons {
        if cart.Total >= c.MinimumTotal &&
            (c.Category == "" || c.Category == cart.Category)
            codes = append(codes, c.Code)
        }
    }
    _, err := fmt.Fprintf(out,
        "To: %s\nCoupons: %v\n", cart.Email, codes)
    return err
}
```

โจทย์ของคุณ

1. แยก Actions / Calculations / Data ให้ครบ
2. ดึง FindApplicableCoupons ออกมา — เขียน table-driven test ก่อน implementation
3. เหลือ SendCouponEmail เป็น action บาง ๆ
4. ทำเสร็จค่อยเปิดดู solution/ ใน repo

กติกาโจทย์: เงินเป็นบาทเต็มไม่ติดลบ · coupon ไม่มีหมวด = ใช้ได้ทุกหมวด · รักษาลำดับเดิม

จำไว้: เป้าหมาย: กติกาธุรกิจทั้งหมดอยู่ใน calculation ที่เทสต์ได้ โดยไม่ต้องแตะ I/O

อ่านต่อ

- Grokking Simplicity — Eric Normand (Manning 2021) · โฟกัส ch.3–5 (ACD), ch.6–7 (immutability), ch.10–14 (first-class functions)
- Functional Programming in Go — Dylan Meeus (Packt 2023)
- Go blog: Range Over Function Types (Go 1.23) — go.dev/blog/range-functions
- Codewalk: First-Class Functions in Go — go.dev/doc/codewalk/functions
- **โค้ดทั้งหมดในวันนี้ (พร้อมเทสต์): ~/src/go-fp — go test ./... ผ่านทุก package**